

.NET Technical Task – Level 1

Estimated time: 4 hours

Goal

Build a small ASP.NET Core Web API where creating a Ticket triggers notifications via different channels (Email, SMS, Push). The task evaluates object-oriented design, dependency injection, error handling, retry logic, and testability.

Technical Requirements

- Use .NET 8 (or .NET 7) ASP.NET Core Web API
- Use in-memory storage (no database required)
- Swagger enabled
- Include unit tests (xUnit or NUnit)

Domain Model

Ticket

- Id: Guid
- Title: string (required, minimum 5 characters)
- Description: string (optional)
- Priority: Low / Medium / High
- CreatedAt: DateTimeOffset

Notification

- Id: Guid
- TicketId: Guid
- Channel: Email / Sms / Push
- Status: Pending / Sent / Failed
- Attempts: int
- LastError: string (optional)
- CreatedAt: DateTimeOffset

Notification Sending

Ticket notifications must be sent through different channels: Email, SMS, and Push. Each channel has its own sending logic and validation rules. Your solution should be designed in a way that:

- Avoids duplicating sending logic
- Allows adding new notification channels in the future with minimal changes to existing code
- Keeps controller logic simple

API Endpoints

- POST /api/tickets - Creates ticket and 3 pending notifications (Email, SMS, Push)

- POST /api/tickets/{id}/notify - Sends all Pending/Failed notifications
- GET /api/tickets/{id} - Returns ticket with notifications

Retry Policy

- If sending fails: increment Attempts
- Set Status = Failed
- Store LastError
- Maximum attempts = 3
- If attempts reached 3 - do not retry again

Idempotency Requirement

- Calling notify endpoint multiple times must NOT create new Notification records
- Only retry existing Pending or Failed notifications

Unit Tests (minimum 6)

- Creating ticket creates 3 notifications in Pending
- Notify sends all and updates statuses correctly
- Retry increments attempts and stops after 3
- Notify is idempotent
- Failure error message is stored in LastError